

KushHR — AWS Server Sizing

Compute, memory, and disk requirements for running KushHR on a single AWS EC2 instance.

Sized for **15–20 users** · Single-instance Docker Compose deployment

At a glance: a **2 vCPU / 8 GB** instance with a **50 GB** disk (e.g. `t3.large` or `t4g.large` + 50 GB gp3) runs KushHR comfortably for 15–20 users. Memory is the resource that matters most — the app runs on top of a full self-hosted database platform, which is what sets the floor.

1 Recommended sizing

Resource	Recommended	Minimum (works, tight)
vCPU (CPU cores)	2 vCPU	2 vCPU
Memory (RAM)	8 GB	4 GB
Disk (EBS gp3)	50 GB	30 GB

Suggested instance type: `t3.large` (x86) or `t4g.large` (ARM / Graviton, lower cost) — both 2 vCPU / 8 GB. Burstable instances suit this workload (low steady-state with occasional bursts). For guaranteed non-burstable CPU, `m6i.large` / `m7g.large` offer the same 2 vCPU / 8 GB.

2 What the server runs

KushHR deploys as a **single instance running a complete self-hosted database platform (Supabase) alongside the application** — the database is bundled, not a separate managed service. One instance therefore runs about **13 containers**:

Database (PostgreSQL 17) · authentication · REST API · realtime · file storage · image proxy · metadata · edge functions · API gateway · connection pooler · admin console · **the KushHR web application** · TLS reverse proxy.

Memory is the binding constraint. Several of these services each hold real memory; the application itself is light, but the platform underneath it sets the floor. The 15–20-user figure has little effect on CPU (concurrency is very low) — it mainly influences data and document growth, i.e. disk.

3 Why these numbers

CPU — 2 cores. 15–20 users generate negligible concurrent load. Two vCPUs comfortably run all ~13 containers plus short bursts. Four cores would be headroom only, not a requirement.

Memory — 8 GB recommended, 4 GB floor. The full platform idles around 3–4 GB and rises under load. 4 GB boots but leaves little room for database caching or for the spikes during a platform version upgrade (many services active at once), creating an out-of-memory risk. 8 GB gives the database caching room, the runtimes headroom, and the operating system a buffer.

Disk — 50 GB recommended, 30 GB floor. Approximate breakdown:

Component	Space
Container images on disk (~13 images; the database image alone is ~2–3 GB)	~10–12 GB
Database data (HR records + append-only audit log, 15–20 users)	~1–2 GB, slow growth
Uploaded documents (contracts / payslips / IDs; 10 MB per-file cap)	budget ~5–10 GB growth
Local backups (encrypted database + storage archives, with retention)	~5–10 GB
OS + Docker overhead + logs + upgrade headroom	~8–10 GB

30 GB runs but is pressured by upgrades and backup retention; 50 GB is the comfortable figure.

4 Two recommendations

1 · Build the application image ahead of time, pull it on the server

Building the app on the instance briefly adds ~1.5–2 GB of memory, which is risky on a 4 GB instance. The planned deployment pulls a prebuilt image from a registry (ECR), removing build memory and CPU from the server entirely. (If the image must be built on the instance instead, size to 8 GB.)

2 · Store backups off the instance (e.g. S3), not on the same disk

Keeping backups on the same disk as the live system means a single disk failure loses both. Moving backups to object storage improves durability and removes ~5–10 GB of growth from the disk figure above.

5 Note

If a Graviton (ARM, `t4g`) instance is chosen for lower cost, the application image must be built for `arm64` in the deployment pipeline. The rest of the platform provides ARM variants, so it is unaffected.

KushHR infrastructure sizing · Single-instance Docker Compose deployment, 15–20 users. Figures are steady-state with modest growth headroom; revisit if user count or document volume grows materially.